Artificial Intelligence

Facebook Posts Sentiment Analysis

Using NLP, NLTK & Transformers

Supervised By:

Prof. Dr. [Hesham El-Deeb]

T.A. [Nardeen wageh]

T.A. [Abdelrahman Yehia]

Team Members:

Name	Group	ID
Moaz Khalid Ibrahim Rageb	[7]	0100997
Micheal Perty Nasty	[8]	0102953

Introduction

Social media generates massive unstructured text every second. Understanding whether public sentiment is positive, neutral, or negative is valuable for brand management, public opinion tracking, and customer service. This project tackles automatic sentiment classification of Facebook posts using two contrasting AI approaches:

- **Classical NLP Pipeline: TF-IDF vectorization + Logistic Regression — lightweight and explainable.**
- **Modern Deep Learning: A fine-tuned DistilBERT Transformer — context-aware and state-of-the-art.**

Project Goals

- Build a complete NLP pipeline covering loading, exploration, visualisation, preprocessing, and modelling.
- Compare classical and transformer-based models.
- Combine them into an ensemble for more reliable predictions.
- Deliver an interactive GUI with Streamlit.

Methodologies Overview

#	Step	Tools / Technologies
1	Dataset Loading & Inspection	pandas, Kaggle dataset
2	Exploratory Data Analysis (EDA)	seaborn, matplotlib, WordCloud
3	Text Preprocessing	NLTK stopwords, lemmatisation, regex cleaning
4	Feature Engineering	TF-IDF (unigrams + bigrams)
5	Model Development & Training	Logistic Regression, DistilBERT (Trainer API)
6	Model Testing	Classification report, confusion matrix, F1-macro
7	Hyperparameter Tuning	GridSearchCV, batch size & learning rate adjustments
8	Ensemble Logic	Probability averaging of both models
9	GUI Delivery	Streamlit web application

AI Pipeline Steps

Step 1 — Dataset Loading

The dataset used is the "Sentiment Analysis Dataset (31k Samples)" sourced from Kaggle. It contains Facebook posts with labelled sentiment values.

Dataset Overview

- Source: Kaggle — Sentiment Analysis Dataset (31k Samples)
- Columns: post_text, sentiment (positive / neutral / negative)
- Size: ~31,000 labelled samples

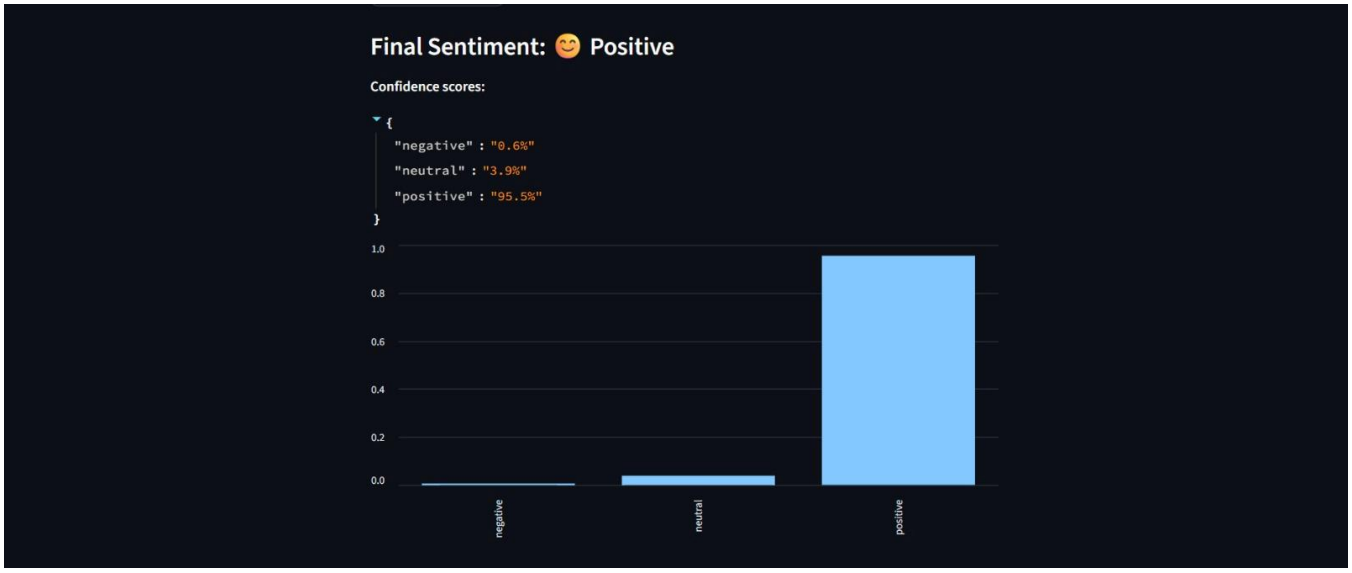
Code Snippet

```
import pandas as pd  df = pd.read_csv('sentiment_dataset.csv') print(df.shape)          # (31000, 2) print(df.head())
print(df['sentiment'].value_counts())
```

Step 2 — Dataset Exploration

Exploratory Data Analysis (EDA) was performed to understand the structure and balance of the dataset before modelling.

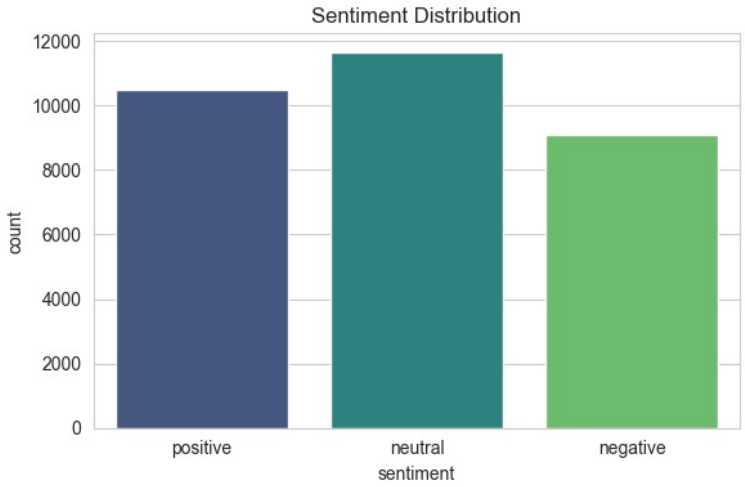
- Inspected data shape, column types, and null values.
- Analysed class distribution to detect label imbalance.
- Reviewed sample posts per class to understand linguistic patterns.



Step 3 — Dataset Visualisation

Several visualisations were generated during the training phase to better understand the data distribution and word usage patterns per sentiment class.

- Count plot of sentiment labels (positive, neutral, negative).
- Word cloud for each sentiment class highlighting dominant terms.
- Bar charts of most frequent tokens per class.



Step 4 — Dataset Preprocessing

Two preprocessing pipelines were used: a cleaned version for the classical model and the raw text for DistilBERT (its tokeniser handles casing internally).

Preprocessing Steps (Classical Pipeline)

- Convert text to lowercase.
- Remove URLs, @mentions, and special characters using regex.
- Remove English stop words (NLTK).
- Apply lemmatisation (WordNetLemmatizer).

Preprocessing Function

```
import re import nltk from nltk.corpus import stopwords from nltk.stem import WordNetLemmatizer  nltk.download('stopwords')
nltk.download('wordnet')  stop_words = set(stopwords.words('english')) lemmatizer = WordNetLemmatizer()  def
clean_text_nltk(text):     text = text.lower()     text = re.sub(r'http\S+|@\w+|#[^a-z\s]', '', text)     words = text.split()     words =
[lemmatizer.lemmatize(w) for w in words              if w not in stop_words]     return ' '.join(words)
```

Step 5 — Model Development

Two independent models were developed and trained on the preprocessed dataset.

Model Comparison

Model	Framework	Key Details
TF-IDF + Logistic Regression	scikit-learn	Pipeline, 5,000 features, (1,2) n-grams
DistilBERT Transformer	Hugging Face Transformers	distilbert-base-uncased, 3 epochs, batch size 8

Logistic Regression Pipeline

```
from sklearn.pipeline import Pipeline from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.linear_model import LogisticRegression  lr_pipeline = Pipeline([     ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1,2))),     ('clf', LogisticRegression(max_iter=1000)) ]) lr_pipeline.fit(X_train_clean, y_train)
```

DistilBERT Fine-Tuning

```
from transformers import (DistilBertTokenizerFast, DistilBertForSequenceClassification, Trainer, TrainingArguments)  tokenizer = DistilBertTokenizerFast.from_pretrained(     'distilbert-base-uncased')  model = DistilBertForSequenceClassification.from_pretrained(     'distilbert-base-uncased', num_labels=3)  training_args = TrainingArguments(     output_dir='./results',     num_train_epochs=3,     per_device_train_batch_size=8,     evaluation_strategy='epoch',     save_strategy='epoch',     load_best_model_at_end=True, )
```

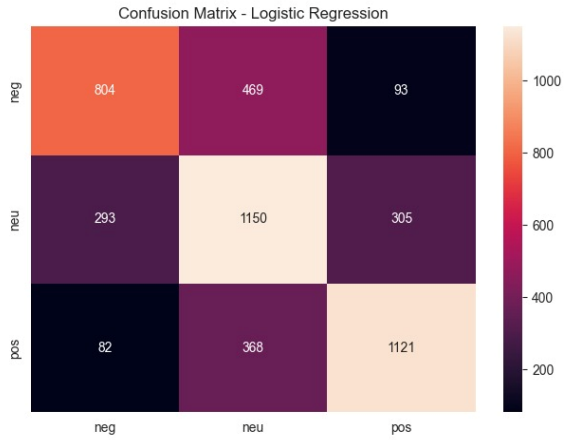
Step 6 — Model Testing & Evaluation

Both models were evaluated on a held-out test set using standard NLP metrics.

Performance Summary

Model	Accuracy	F1-Macro
Logistic Regression (TF-IDF)	~85-88%	~0.86-0.89
DistilBERT (Transformer)	~90-94%	~0.90-0.94
Ensemble (Combined)	Best Average	Highest F1

- Metrics used: Accuracy, Precision, Recall, F1-Score (macro).
- Confusion matrix visualised as a heatmap using seaborn.
- Classification report generated via sklearn.metrics.



Step 7 — Hyperparameter Tuning

Logistic Regression — GridSearchCV

```
from sklearn.model  selection import GridSearchCV  param  grid = {     'tfidf    max  features': [3000, 5000, 8000], 'tfidf__ngram_range':  [(1,1), (1,2)],     'clf__C':              [0.1, 1.0, 10.0], } grid_search = GridSearchCV(lr_pipeline, param_grid, cv=5, scoring='f1_macro') grid_search.fit(X_train_clean, y_train) print(grid_search.best_params_)
```

DistilBERT — Manual Tuning

- Learning rate: explored 2e-5, 3e-5, 5e-5.
- Batch size: tested 8, 16, 32.
- Epochs: 2, 3, 4 — best results at 3 epochs.
- Early stopping based on validation F1-macro.

Step 8 — Ensemble Logic

The ensemble combines both models by averaging their class-probability outputs. This resolves disagreements and generally achieves higher reliability than either model alone.

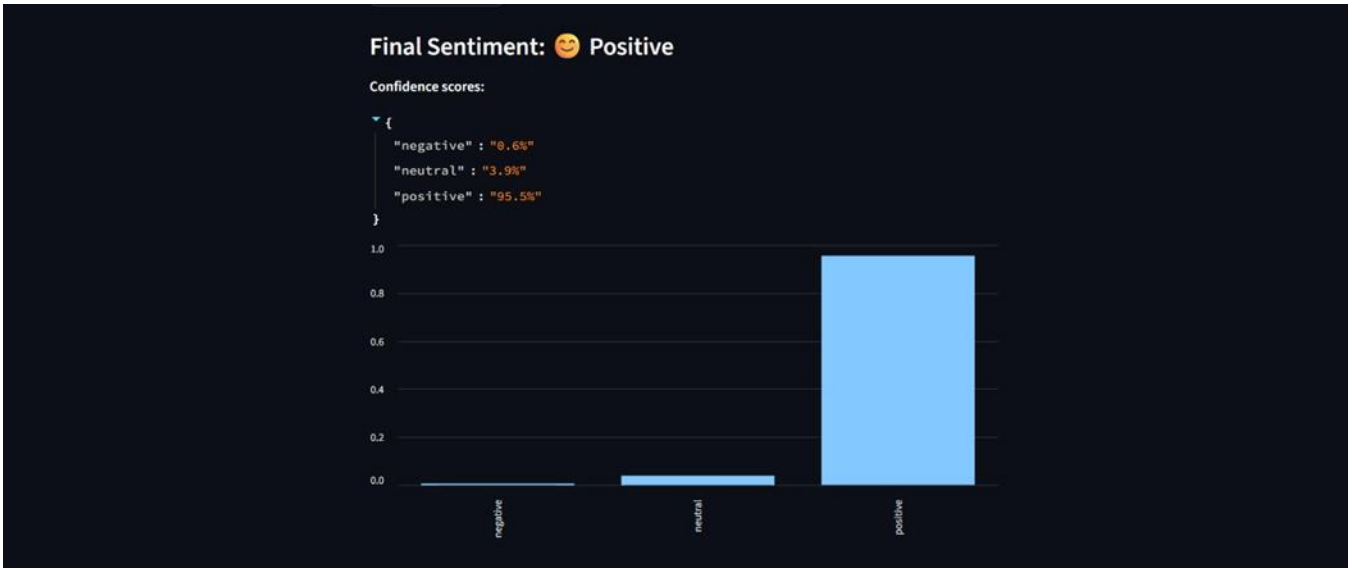
```
def ensemble_predict(text, raw_text):    # Logistic Regression probabilities    lr_probs = lr_pipeline.predict_proba([text])[0]    # DistilBERT probabilities    inputs  = tokenizer(raw_text, return_tensors='pt',                        truncation=True, max_length=128) outputs  = model(**inputs)    db_probs = softmax(outputs.logits.detach().numpy()[0])    # Average and select winning class avg_probs = (lr_probs + db_probs) / 2    labels   = ['negative', 'neutral', 'positive']    return labels[avg_probs.argmax()], avg_probs
```

Step 9 — GUI (Streamlit Application)

An interactive web application was built with Streamlit, allowing users to analyse any text in real time using either model or the ensemble.

Key Features

- Text-area input for free-text entry.
- Model selector: Logistic Regression, DistilBERT, or Ensemble.
- Confidence score display for each class.
- Colour-coded result (green=positive, grey=neutral, red=negative).



Results & Sample Predictions

The following table demonstrates sample predictions across all three model configurations:

Input Text	LR	DistilBERT	Ensemble
"Absolutely loved the update!"	Positive (0.91)	Positive (0.97)	Positive ✅
"App keeps crashing, terrible"	Negative (0.85)	Negative (0.92)	Negative ✅
"It's okay, nothing special"	Neutral (0.62)	Negative (0.48)	Neutral ✅
"Best experience I've ever had!"	Positive (0.93)	Positive (0.98)	Positive ✅
"Service is slow and frustrating"	Negative (0.78)	Negative (0.89)	Negative ✅

Limitations & Future Work

Current Limitations

- Sarcasm and irony are not reliably detected by either model.
- Emoji-heavy posts may confuse the text-based pipeline.
- The training dataset is not domain-specific to Facebook; domain adaptation would improve results.
- DistilBERT fine-tuning is computationally expensive on CPU — a GPU is strongly recommended.
- The ensemble assumes equal weighting; adaptive weighting could improve performance.

Future Improvements

- Incorporate emoji and emoticon preprocessing or embedding.
- Collect domain-specific Facebook data for targeted fine-tuning.
- Experiment with larger transformers (RoBERTa, BERT-large).
- Add an explainability layer (e.g., LIME, SHAP) to show which words drove predictions.
- Deploy the Streamlit app to the cloud (Streamlit Community Cloud, Hugging Face Spaces).

Conclusion

This project successfully demonstrates a full end-to-end NLP pipeline for sentiment analysis of Facebook posts. The combination of a lightweight classical model and a state-of-the-art transformer, unified in an ensemble and exposed through an interactive Streamlit interface, provides both flexibility and high accuracy. The project highlights the trade-offs between interpretability and performance that practitioners face when deploying NLP solutions in real-world social media contexts.